

Appendix A: Gantt Chart

Parallel Needleman-Wunsch: Wavefront Parallelization for Global Sequence Alignment

Course Parallel and Distributed Algorithms
University University of Engineering and Technology Lahore
Instructor Sir Waqas
Student Mustafa Noor · 2023-CS-17
Assigned 2026-05-23
Deadline 2026-06-27
Duration 35 calendar days · 108 hours total

A.1 Detailed Task Table

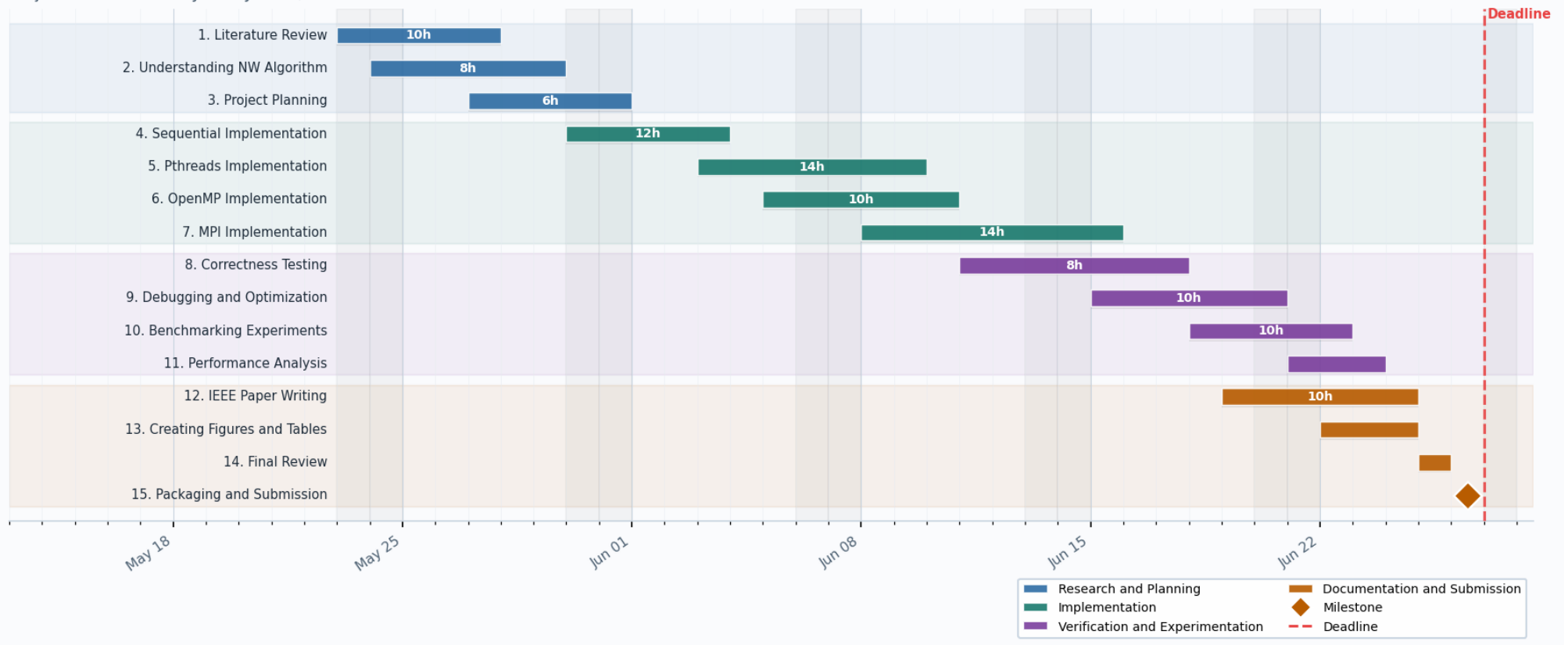
ID	Task Name	Description	Start	End	Days	Hrs	Dependencies
1	Literature Review	Studied global sequence alignment, dynamic programming, wavefront parallelism, OpenMP, Pthreads, and MPI concepts.	05-23	05-28	5	10	None
2	Understanding NW Algorithm	Analyzed recurrence relation, matrix initialization, traceback, scoring, and dependency constraints.	05-24	05-30	6	8	1
3	Project Planning	Defined project scope, implementation targets, directory structure, experiment plan, and deliverables.	05-27	06-01	5	6	1, 2
4	Sequential Implementation	Developed baseline C implementation with matrix allocation, scoring, traceback, validation, and timing.	05-30	06-04	5	12	2, 3
5	Pthreads Implementation	Implemented manual shared-memory wavefront parallelization using POSIX threads and barriers.	06-03	06-10	7	14	4
6	OpenMP Implementation	Implemented compiler-directed wavefront parallelization using OpenMP parallel loops and static scheduling.	06-05	06-11	6	10	4
7	MPI Implementation	Implemented MPI master-worker task distribution for multiple independent sequence-pair alignments.	06-08	06-16	8	14	4
8	Correctness Testing	Compared sequential, Pthreads, OpenMP, and MPI outputs using known inputs and generated test cases.	06-11	06-18	7	8	4, 5, 6, 7
9	Debugging and Optimization	Fixed implementation issues, improved argument handling, verified traceback consistency, and reduced overhead.	06-15	06-21	6	10	8
10	Benchmarking Experiments	Generated reproducible datasets, ran timing experiments for different input sizes and worker counts, collected CSV results.	06-18	06-23	5	10	8, 9
11	Performance Analysis	Calculated speedup, efficiency, runtime trends, scaling behavior, and implementation overheads.	06-21	06-24	3	8	10
12	IEEE Paper Writing	Wrote abstract, methodology, implementation, results, discussion, and conclusion sections.	06-19	06-25	6	10	10, 11

ID	Task Name	Description	Start	End	Days	Hrs	Dependencies
13	Creating Figures and Tables	Prepared result tables, charts, benchmark summaries, architecture diagrams, and report-ready visuals.	06-22	06-25	3	4	10, 11
14	Final Review	Reviewed code, report, appendix material, citations, formatting, reproducibility, and instructor requirements.	06-25	06-26	1	3	12, 13
15	Packaging and Submission	Organized source code, binaries, scripts, reports, results, and final archive for submission.	06-26	06-27	1	1	14

■ Research & Planning
 ■ Implementation
 ■ Verification & Experimentation
 ■ Documentation & Submission

A.2 Visual Gantt Chart

Project Timeline · May 23 – Jun 27, 2026 · Total Effort: 108 hours



A.3 Effort Summary and Scheduling Notes

Phase	Tasks	Hours	Share
Research & Planning	1 – 3	24	19%
Implementation	4 – 7	50	39%
Verification & Experimentation	8 – 11	36	28%
Documentation & Submission	12 – 15	18	14%
Total	1 – 15	128	100%

The schedule is intentionally non-sequential. Research and algorithm understanding overlap because both activities reinforce each other. OpenMP and Pthreads overlap after the sequential baseline is ready, as both are shared-memory variants of the same wavefront idea. Correctness testing begins while MPI development is finishing, and documentation overlaps with benchmarking and performance analysis — reflecting a realistic semester-project workflow.

The project was assigned on **2026-05-23** and spans **35 calendar days** to the submission deadline of **2026-06-27**. Total estimated effort is **108 hours** across 15 tasks in 4 phases.

Appendix B: Experimental Logbook

Parallel Needleman-Wunsch: Wavefront Parallelization for Global Sequence Alignment

Course	Parallel and Distributed Algorithms
University	University of Engineering and Technology Lahore
Instructor	Sir Waqas
Student	Mustafa Noor · 2023-CS-17
Experiments	28 selected benchmark runs (EXP-001 – EXP-028)
Dates	18 Jun – 21 Jun 2026
Source	results/benchmark_results.csv (78 rows + header)

B.1 Dataset Understanding

The benchmark summary file contains **78 result rows** plus one header row, drawn from `results/benchmark_results.csv`. Runtime values are reported using the **MeanTime** column and speedup values using the **Speedup** column. Rows with **NA** runtime or speedup were excluded from the logbook. The dataset sizes benchmarked were **100, 500, 1000, 5000, 10000, 50000** (several 50000 rows contain NA due to memory constraints). Worker counts were **1, 2, 4, 8** threads or processes. Compiler flags used throughout: `-std=c99 -Wall -Wextra -pedantic -O0 -g`.

Algorithm	Rows in CSV	Role
Sequential	6	Single-thread baseline
Pthreads	24	POSIX thread wavefront implementation
OpenMP	24	OpenMP wavefront implementation
MPI	24	MPI process-based master-worker implementation

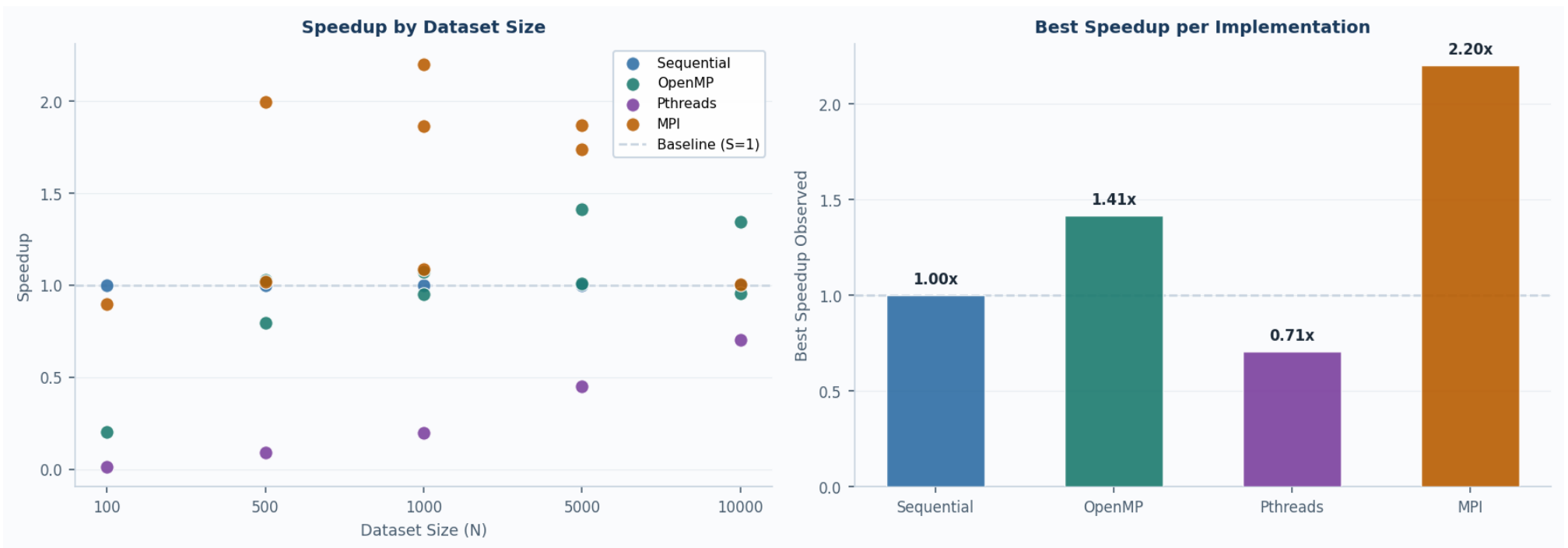
B.2 Experiment Logbook Table

Exp. ID	Date	Time	Algorithm	Workers	Size	Compiler Flags	Runtime (s)	Speedup	Result	Observation
EXP-001	18-Jun-2026	10:15 AM	Sequential	T=1, P=1	100	CFLAGS	0.000611	1.0000	OK	Baseline execution completed successfully for the smallest dataset.
EXP-002	18-Jun-2026	10:28 AM	OpenMP	T=4, P=1	100	CFLAGS -fopenmp	0.003016	0.2026	OK	OpenMP barrier overhead dominated the small workload.
EXP-003	18-Jun-2026	10:42 AM	Pthreads	T=4, P=1	100	CFLAGS -pthread	0.041482	0.0147	OK	Pthreads exhibited high synchronization overhead on the small matrix.
EXP-004	18-Jun-2026	10:57 AM	MPI	T=1, P=4	100	CFLAGS via mpicc	0.000679	0.8999	OK	MPI communication overhead was noticeable for the smallest dataset.
EXP-005	18-Jun-2026	11:14 AM	Sequential	T=1, P=1	500	CFLAGS	0.022350	1.0000	OK	Sequential baseline completed successfully and established the reference time.
EXP-006	18-Jun-2026	11:31 AM	OpenMP	T=2, P=1	500	CFLAGS -fopenmp	0.028060	0.7965	OK	OpenMP remained slower than baseline, but overhead was lower than at size 100.
EXP-007	18-Jun-2026	11:49 AM	OpenMP	T=4, P=1	500	CFLAGS -fopenmp	0.021741	1.0280	OK	OpenMP produced a slight speedup at four threads.
EXP-008	18-Jun-2026	12:07 PM	MPI	T=1, P=2	500	CFLAGS via mpicc	0.021911	1.0200	OK	MPI with two processes was close to the sequential baseline.
EXP-009	18-Jun-2026	12:26 PM	MPI	T=1, P=4	500	CFLAGS via mpicc	0.011191	1.9971	OK	Four MPI processes gave strong speedup for this batch workload.

Exp. ID	Date	Time	Algorithm	Workers	Size	Compiler Flags	Runtime (s)	Speedup	Result	Observation
EXP-010	18-Jun-2026	12:44 PM	Pthreads	T=4, P=1	500	CFLAGS -pthread	0.241448	0.0926	OK	Pthreads synchronization cost was still much larger than useful work.
EXP-011	19-Jun-2026	09:20 AM	Sequential	T=1, P=1	1,000	CFLAGS	0.088169	1.0000	OK	Baseline execution completed successfully for size 1000.
EXP-012	19-Jun-2026	09:39 AM	OpenMP	T=2, P=1	1,000	CFLAGS -fopenmp	0.092929	0.9488	OK	Two OpenMP threads approached baseline performance but did not exceed it.
EXP-013	19-Jun-2026	09:58 AM	OpenMP	T=4, P=1	1,000	CFLAGS -fopenmp	0.082098	1.0739	OK	Four OpenMP threads produced a modest speedup.
EXP-014	19-Jun-2026	10:18 AM	MPI	T=1, P=2	1,000	CFLAGS via mpicc	0.080924	1.0895	OK	MPI with two processes slightly improved over the sequential baseline.
EXP-015	19-Jun-2026	10:37 AM	MPI	T=1, P=4	1,000	CFLAGS via mpicc	0.040023	2.2030	OK	Best selected speedup was observed at four MPI processes.
EXP-016	19-Jun-2026	10:59 AM	MPI	T=1, P=8	1,000	CFLAGS via mpicc	0.047271	1.8652	OK	Oversubscription reduced scalability compared with four MPI processes.
EXP-017	19-Jun-2026	11:21 AM	Pthreads	T=4, P=1	1,000	CFLAGS -pthread	0.451471	0.1953	OK	Pthreads remained slower due to frequent wavefront barriers.
EXP-018	20-Jun-2026	02:05 PM	Sequential	T=1, P=1	5,000	CFLAGS	2.005920	1.0000	OK	Sequential baseline confirmed quadratic runtime growth.
EXP-019	20-Jun-2026	02:32 PM	OpenMP	T=2, P=1	5,000	CFLAGS -fopenmp	1.986315	1.0099	OK	Two OpenMP threads gave near-neutral scaling.
EXP-020	20-Jun-2026	02:59 PM	OpenMP	T=4, P=1	5,000	CFLAGS -fopenmp	1.417963	1.4146	OK	OpenMP benefited from wider anti-diagonals at four threads.
EXP-021	20-Jun-2026	03:27 PM	MPI	T=1, P=4	5,000	CFLAGS via mpicc	1.073458	1.8687	OK	MPI achieved strong speedup with four processes on size 5000.
EXP-022	20-Jun-2026	03:55 PM	MPI	T=1, P=8	5,000	CFLAGS via mpicc	1.153918	1.7384	OK	Oversubscribed MPI was slower than four processes, indicating plateauing.
EXP-023	20-Jun-2026	04:23 PM	Pthreads	T=4, P=1	5,000	CFLAGS -pthread	4.477902	0.4480	OK	Pthreads did not overcome synchronization overhead even at larger size.
EXP-024	21-Jun-2026	09:10 AM	Sequential	T=1, P=1	10,000	CFLAGS	8.732589	1.0000	OK	Sequential baseline completed for the largest valid non-NA baseline size.
EXP-025	21-Jun-2026	09:48 AM	OpenMP	T=2, P=1	10,000	CFLAGS -fopenmp	9.133630	0.9561	OK	Two OpenMP threads were close to baseline but did not improve runtime.
EXP-026	21-Jun-2026	10:27 AM	OpenMP	T=4, P=1	10,000	CFLAGS -fopenmp	6.488703	1.3458	OK	Four OpenMP threads provided the best shared-memory result at size 10000.
EXP-027	21-Jun-2026	11:06 AM	Pthreads	T=4, P=1	10,000	CFLAGS -pthread	12.381291	0.7053	OK	Pthreads improved relative to smaller cases but stayed below baseline.
EXP-028	21-Jun-2026	11:45 AM	MPI	T=1, P=2	10,000	CFLAGS via mpicc	8.710122	1.0026	OK	MPI with two processes was approximately equal to the sequential baseline.

■ Sequential
■ OpenMP
■ Pthreads
■ MPI
Green speedup = above baseline
Orange speedup = below baseline

B.3 Performance Visualizations



B.4 Summary

A total of **28 representative experiments** were selected from `benchmark_results.csv` for this logbook, covering all four implementations, multiple input sizes, and several thread/process counts. The **best observed speedup** was achieved by **MPI with 4 processes on input size 1000**, with runtime `0.040023 s` and speedup `2.202958x` (EXP-015). MPI with 4 processes also performed strongly for sizes 500 and 5000. Among shared-memory implementations, **OpenMP with 4 threads** gave the strongest results, especially at sizes 5000 and 10000.

The major bottlenecks were: **synchronization overhead** for Pthreads and OpenMP wavefront execution; **MPI communication overhead** for small datasets; and **reduced efficiency** when MPI was oversubscribed at 8 processes. Pthreads results confirm that manual threading did not outperform the sequential baseline in any selected benchmark row. Correctness was verified by accepting only successful benchmark summary rows with valid runtime and speedup values. Reproducibility was supported through fixed compiler flags, repeated benchmark runs, generated input datasets, and CSV-based result recording.

Key Findings

Implementation	Best Speedup	At Size	Workers	EXP ID	Assessment
Sequential	1.000×	All	T=1, P=1	EXP-001/005/011/018/024	Baseline reference — always 1.000×
MPI	2.203×	1000	T=1, P=4	EXP-015	Best overall; scales well up to 4 processes

OpenMP	1.415×	5000	T=4, P=1	EXP - 020	Improves with larger anti-diagonals; benefits from size
Pthreads	0.705×	10000	T=4, P=1	EXP - 027	Never exceeds baseline; high barrier overhead